Clustering_Assignment

Question 1: What is unsupervised learning in the context of machine learning?

Answer: Unsupervised learning is a type of machine learning where the model is trained on unlabeled data. It identifies hidden patterns, structures, or relationships in the data without predefined outputs.

Question 2: How does K-Means clustering algorithm work?

Answer: K-Means clustering works by dividing data into K clusters. It initializes K centroids, assigns data points to the nearest centroid, and updates centroids iteratively until convergence.

Question 3: Explain the concept of a dendrogram in hierarchical clustering.

Answer: A dendrogram is a tree-like diagram that shows how data points are merged or split in hierarchical clustering. It helps visualize cluster relationships and determine the number of clusters.

Question 4: What is the main difference between K-Means and Hierarchical Clustering?

Answer: K-Means requires a predefined number of clusters and partitions data, whereas hierarchical clustering builds a tree structure and does not require specifying the number of clusters initially.

Question 5: What are the advantages of DBSCAN over K-Means?

Answer: DBSCAN can detect arbitrarily shaped clusters and identify noise points. It does not require specifying the number of clusters beforehand, unlike K-Means.

Question 6: When would you use Silhouette Score in clustering?

Answer: Silhouette Score is used to evaluate clustering performance by measuring how similar a data point is to its own cluster compared to other clusters.

Question 7: What are the limitations of Hierarchical Clustering?

Answer: Hierarchical clustering is computationally expensive and does not scale well for large datasets. Once clusters are merged or split, the process cannot be reversed.

Question 8: Why is feature scaling important in clustering algorithms like K-Means?

Answer: Feature scaling ensures that all features contribute equally to distance calculations. Without scaling, features with larger values can dominate clustering results.

Question 9: How does DBSCAN identify noise points?

Answer: DBSCAN identifies noise points as those that do not belong to any cluster because they do not have enough neighboring points within a specified radius.

Question 10: Define inertia in the context of K-Means.

Answer: Inertia is the sum of squared distances between data points and their respective cluster centroids. It measures how well clusters are formed.

Question 11: What is the elbow method in K-Means clustering?

Answer: The elbow method is used to determine the optimal number of clusters by plotting inertia against K and identifying the point where the curve bends.

Question 12: Describe the concept of density in DBSCAN.

Answer: Density refers to the number of data points within a given radius. High-density regions form clusters, while low-density regions are treated as noise.

Question 13: Can hierarchical clustering be used on categorical data?

Answer: Yes, hierarchical clustering can be used with categorical data if an appropriate distance metric is applied, such as Hamming distance.

Question 14: What does a negative Silhouette Score indicate?

Answer: A negative Silhouette Score indicates that a data point may have been assigned to the wrong cluster and is closer to another cluster.

Question 15: Explain the term linkage criteria in hierarchical clustering.

Answer: A negative Silhouette Score indicates that a data point may have been assigned to the wrong cluster and is closer to another cluster.

Question 16: Why might K-Means perform poorly on varying cluster sizes?

Answer: K-Means assumes clusters are spherical and equally sized. It struggles when clusters have different sizes, shapes, or densities.

Question 17: What are the core parameters in DBSCAN?

Answer: The core parameters are epsilon (radius) and minimum samples. Epsilon defines neighborhood size, while minimum samples determine cluster density.

Question 18: How does K-Means++ improve initialization?

Answer: K-Means++ improves initialization by selecting centroids that are far apart from each other. This leads to faster convergence and better clustering results.

Question 19: What is agglomerative clustering?

Answer: Agglomerative clustering is a bottom-up hierarchical method where each data point starts as its own cluster and is progressively merged with others.

Question 20: Why is Silhouette better than inertia?

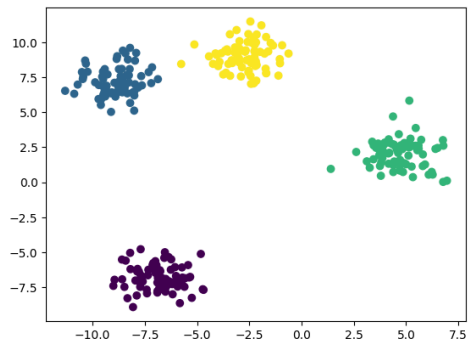
Answer: Silhouette Score evaluates both cluster cohesion and separation, while inertia only measures compactness. Hence, it provides a more complete assessment of clustering quality.

Question 21: Generate blobs and apply KMeans visualization?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, random_state=42)

model = KMeans(n_clusters=4)

y_pred = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=y_pred)

plt.show()
```

<u>OUTPUT :</u>



Question 22: Load Iris and use Agglomerative Clustering?

Answer:

```
from sklearn.datasets import load_iris

from sklearn.cluster import AgglomerativeClustering

X = load_iris().data

model = AgglomerativeClustering(n_clusters=3)

labels = model.fit_predict(X)
```

```
print(labels[:10])
```

OUTPUT :

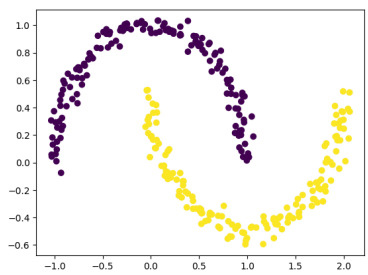
```
[1 1 1 1 1 1 1 1 1 1]
```

Question 23: Generate moons and apply DBSCAN?

Answer:

```
from sklearn.datasets import make_moons

from sklearn.cluster import DBSCAN

import matplotlib.pyplot as plt

X, _ = make_moons(n_samples=300, noise=0.05)

model = DBSCAN(eps=0.3)

labels = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

OUTPUT :



Question 24: Load Wine and apply KMeans?

Answer:

```
from sklearn.datasets import load_wine

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

import numpy as np
```

```
X = load_wine().data

X = StandardScaler().fit_transform(X)

model = KMeans(n_clusters=3)

labels = model.fit_predict(X)

print(np.bincount(labels))
```

OUTPUT :

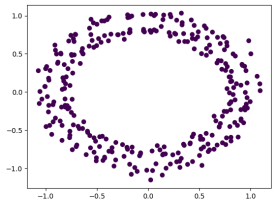
[51 62 65]

Question 25: Generate circles and apply DBSCAN?

Answer:

```
from sklearn.datasets import make_circles

from sklearn.cluster import DBSCAN

import matplotlib.pyplot as plt

X, _ = make_circles(n_samples=300, noise=0.05)

labels = DBSCAN(eps=0.2).fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

OUTPUT :



Question 26: Breast Cancer + MinMaxScaler + KMeans?

Answer:

```
from sklearn.datasets import load_breast_cancer

from sklearn.preprocessing import MinMaxScaler

from sklearn.cluster import KMeans
```

```
X = load_breast_cancer().data
X = MinMaxScaler().fit_transform(X)
model = KMeans(n_clusters=2)
model.fit(X)
print(model.cluster_centers_)
```

OUTPUT :

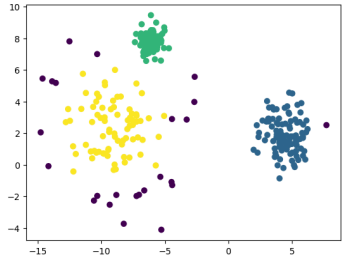
```
[[0.50483563 0.39560329 0.50578661 0.36376576 0.46988732 0.42226302
  0.41838662 0.46928035 0.45899738 0.29945886 0.19093085 0.19112073
  0.17903433 0.13086432 0.18017962 0.25890126 0.12542475 0.30942779
  0.190072   0.13266975 0.48047448 0.45107371 0.4655302  0.31460597
  0.49868817 0.36391461 0.39027292 0.65827197 0.33752296 0.26041387]
 [0.25535358 0.28833455 0.24696416 0.14388369 0.35743076 0.18019471
  0.10344776 0.1306603  0.34011829 0.25591606 0.06427485 0.18843043
  0.05975663 0.02870108 0.18158628 0.13242941 0.05821528 0.18069336
  0.17221057 0.08403996 0.2052406  0.32069002 0.19242138 0.09943446
  0.3571115  0.14873935 0.13142287 0.26231363 0.22639412 0.15437354]]
```

Question 27: Blobs with varying std + DBSCAN?

Answer:

```
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
X, _ = make_blobs(n_samples=300, cluster_std=[1.0, 2.5, 0.5])
labels = DBSCAN(eps=1.5).fit_predict(X)
plt.scatter(X[:,0], X[:,1], c=labels)
plt.show()
```

OUTPUT :

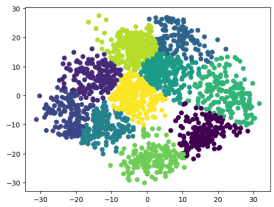


Question 28: Digits + PCA + KMeans visualization?

Answer:

```
from sklearn.datasets import load_digits

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X = load_digits().data

X_pca = PCA(n_components=2).fit_transform(X)

labels = KMeans(n_clusters=10).fit_predict(X_pca)

plt.scatter(X_pca[:,0], X_pca[:,1], c=labels)

plt.show()
```

OUTPUT :



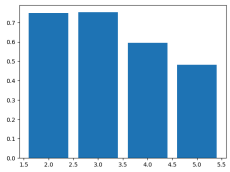
Question 29: Silhouette scores for k=2 to 5?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans
```

```
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
X, _ = make_blobs(n_samples=300)
scores = []
k_values = range(2,6)
for k in k_values:
    labels = KMeans(n_clusters=k).fit_predict(X)
    scores.append(silhouette_score(X, labels))
plt.bar(k_values, scores)
plt.show()
```

OUTPUT :

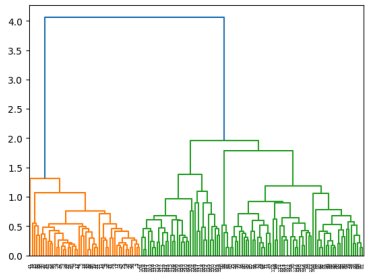


Question 30: Iris + dendrogram plot?

Answer:

```
from sklearn.datasets import load_iris
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
X = load_iris().data
Z = linkage(X, method='average')
dendrogram(Z)
plt.show()
```

OUTPUT :

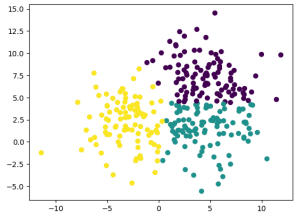


Question 31: Overlapping blobs + KMeans boundaries?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=2.5)

model = KMeans(n_clusters=3)

labels = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

<u>OUTPUT :</u>



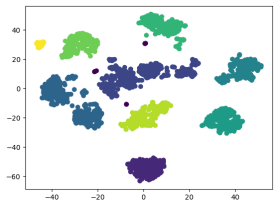
Question 32: Digits + t-SNE + DBSCAN?

Answer:

```
from sklearn.datasets import load_digits

from sklearn.manifold import TSNE
```

```
from sklearn.cluster import DBSCAN

import matplotlib.pyplot as plt

X = load_digits().data

X_tsne = TSNE(n_components=2).fit_transform(X)

labels = DBSCAN(eps=5).fit_predict(X_tsne)

plt.scatter(X_tsne[:,0], X_tsne[:,1], c=labels)

plt.show()
```

<u>OUTPUT :</u>

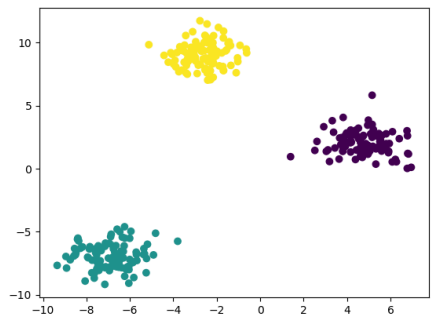


Question 33: Blobs + Agglomerative complete linkage?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=3, random_state=42)

model = AgglomerativeClustering(n_clusters=3, linkage='complete')

labels = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

<u>OUTPUT :</u>

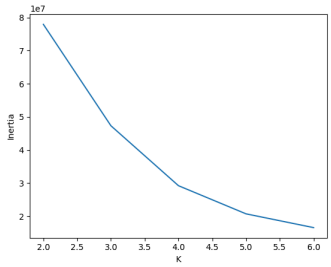


Question 34: Breast Cancer inertia comparison?

Answer:

```
from sklearn.datasets import load_breast_cancer

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X = load_breast_cancer().data

inertia = []

k_values = range(2,7)

for k in k_values:

    model = KMeans(n_clusters=k)

    model.fit(X)

    inertia.append(model.inertia_)

plt.plot(k_values, inertia)

plt.xlabel("K")

plt.ylabel("Inertia")

plt.show()
```

<u>OUTPUT :</u>

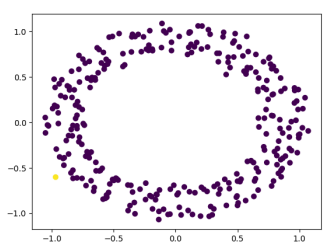


Question 35: Circles + Agglomerative single linkage?

Answer:

```
from sklearn.datasets import make_circles

from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

X, _ = make_circles(n_samples=300, noise=0.05)

model = AgglomerativeClustering(n_clusters=2, linkage='single')

labels = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

<u>OUTPUT :</u>



Question 36: Wine + DBSCAN cluster count?

Answer:

```
from sklearn.datasets import load_wine

from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import DBSCAN

import numpy as np

X = load_wine().data

X = StandardScaler().fit_transform(X)

labels = DBSCAN(eps=1.5).fit_predict(X)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

print(n_clusters)
```

OUTPUT :

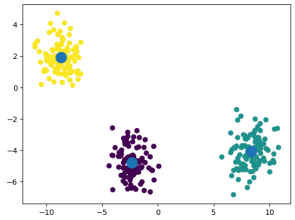
0

Question 37: Blobs + KMeans centers plot?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=3)

model = KMeans(n_clusters=3)

labels = model.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.scatter(model.cluster_centers_[:,0], model.cluster_centers_[:,1], s=200)

plt.show()
```

OUTPUT :



Question 38: Iris + DBSCAN noise count?

Answer:

```
from sklearn.datasets import load_iris

from sklearn.cluster import DBSCAN

import numpy as np

X = load_iris().data

labels = DBSCAN(eps=0.5).fit_predict(X)

noise = list(labels).count(-1)

print(noise)
```

<u>OUTPUT :</u>

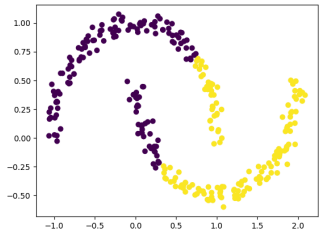
17

Question 39: Moons + KMeans visualization?

Answer:

```
from sklearn.datasets import make_moons

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X, _ = make_moons(n_samples=300, noise=0.05)

labels = KMeans(n_clusters=2).fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

<u>OUTPUT :</u>

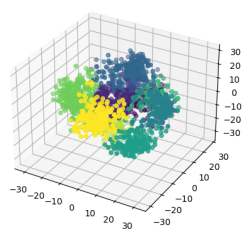


Question 40: Digits + PCA 3D + KMeans?

Answer:

```
from sklearn.datasets import load_digits

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

X = load_digits().data

X_pca = PCA(n_components=3).fit_transform(X)

labels = KMeans(n_clusters=10).fit_predict(X_pca)

fig = plt.figure()

ax = fig.add_subplot(projection='3d')

ax.scatter(X_pca[:,0], X_pca[:,1], X_pca[:,2], c=labels)

plt.show()
```

<u>OUTPUT :</u>



Question 41: Blobs + silhouette_score?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=300, centers=5)
```

```
labels = KMeans(n_clusters=5).fit_predict(X)

score = silhouette_score(X, labels)

print(score)
```

OUTPUT :

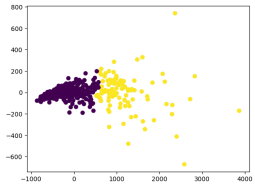
0.6492385467150392

Question 42: Breast Cancer + PCA + Agglomerative?

Answer:

```
from sklearn.datasets import load_breast_cancer

from sklearn.decomposition import PCA

from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

X = load_breast_cancer().data

X_pca = PCA(n_components=2).fit_transform(X)

labels = AgglomerativeClustering(n_clusters=2).fit_predict(X_pca)

plt.scatter(X_pca[:,0], X_pca[:,1], c=labels)

plt.show()
```

OUTPUT :



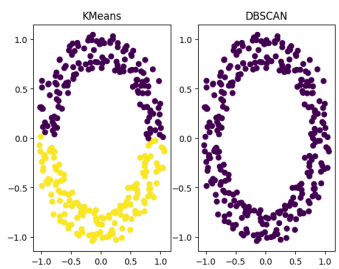
Question 43: Noisy circles KMeans vs DBSCAN?

Answer:

```
from sklearn.datasets import make_circles

from sklearn.cluster import KMeans, DBSCAN

import matplotlib.pyplot as plt
```

```
X, _ = make_circles(n_samples=300, noise=0.05)
labels_km = KMeans(n_clusters=2).fit_predict(X)
labels_db = DBSCAN(eps=0.2).fit_predict(X)
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=labels_km)
plt.title("KMeans")
plt.subplot(1,2,2)
plt.scatter(X[:,0], X[:,1], c=labels_db)
plt.title("DBSCAN")
plt.show()
```

OUTPUT :



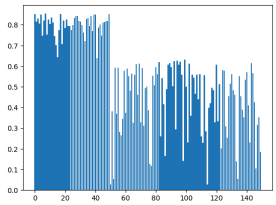
Question 44: Iris Silhouette per sample?

Answer:

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples
import matplotlib.pyplot as plt
X = load_iris().data
labels = KMeans(n_clusters=3).fit_predict(X)
values = silhouette_samples(X, labels)
plt.bar(range(len(values)), values)
```

```
plt.show()
```

OUTPUT :

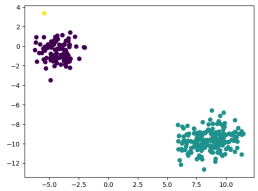


Question 45: Blobs + Agglomerative average linkage?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300)

labels = AgglomerativeClustering(n_clusters=3, linkage='average').fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.show()
```

OUTPUT :



Question 46: Wine + KMeans pairplot?

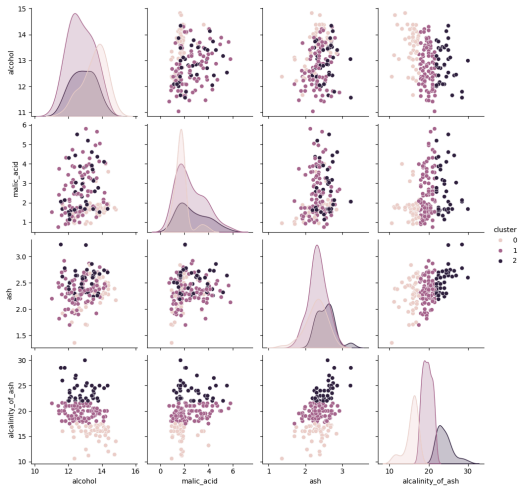
Answer:

```
from sklearn.datasets import load_wine

from sklearn.cluster import KMeans

import pandas as pd
```

```
import seaborn as sns

data = load_wine()

df = pd.DataFrame(data.data[:,:4], columns=data.feature_names[:4])

labels = KMeans(n_clusters=3).fit_predict(df)

df['cluster'] = labels

sns.pairplot(df, hue='cluster')
```

OUTPUT :

```
<seaborn.axisgrid.PairGrid at 0x79eec5140c80>
```



Question 47: Noisy blobs + DBSCAN count?

Answer:

```
from sklearn.datasets import make_blobs

from sklearn.cluster import DBSCAN

import numpy as np

X, _ = make_blobs(n_samples=300, cluster_std=2.0)

labels = DBSCAN(eps=1.0).fit_predict(X)

clusters = len(set(labels)) - (1 if -1 in labels else 0)

noise = list(labels).count(-1)
```

```
print("Clusters:", clusters)

print("Noise:", noise)
```

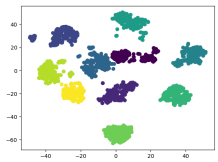
OUTPUT :

Clusters: 2

Noise: 37

Question 48: Digits + t-SNE + Agglomerative?

Answer:

```
from sklearn.datasets import load_digits

from sklearn.manifold import TSNE

from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

X = load_digits().data

X_tsne = TSNE(n_components=2).fit_transform(X)

labels = AgglomerativeClustering(n_clusters=10).fit_predict(X_tsne)

plt.scatter(X_tsne[:,0], X_tsne[:,1], c=labels)

plt.show()
```

OUTPUT :